

Medical Image Anomaly Localization Using AA-CLIP: Adapting Vision-Language Models for Zero-Shot Colonoscopy Polyp Localization

Mohit Patle 27PGAI0102 Jinali Shah 27PGAI0002
Kaushik Gadipelly 27PGAI0074

PGP in Artificial Intelligence and Data Science, 2026–2027, Jio Institute

Guides: Dr. Sudipta Roy, Ms. Aazin Sheikh, Ms. Priyanka Mishra

Abstract

Finding polyps in colonoscopy videos by hand is slow: a doctor or trained annotator has to mark the exact area of every image, and this has to be done again for every new hospital or camera. We tested whether a large pretrained model called CLIP, which already understands pictures and words together, could do this job with no new labels at all. On its own, CLIP turns out to be bad at this: it knows what a colon looks like, but it cannot reliably tell a polyp apart from healthy tissue. We reproduced a recent method called AA-CLIP (CVPR 2025), which fixes this by training two small add-on modules while leaving CLIP itself completely untouched. We ran a fair before-and-after test: we scored plain CLIP on three colonoscopy datasets (CVC-ClinicDB, CVC-ColonDB, Kvasir-SEG), trained the two add-on modules using only the ClinicDB dataset, and then scored the trained model on all three datasets using the exact same scoring code both times. On the two datasets the model had never seen before, pixel-level AUROC went from 67.6–71.0 up to 92.8–97.2, and pixel-level Average Precision went from 22.5–34.5 up to 71.6–90.7, more than tripling. Looking at the actual heatmaps, the untrained model lights up bright spots and edges that are not polyps, while the trained model settles right on the polyp. We also list the honest limits of this study, such as the ClinicDB score being measured on training images, only one run being performed, and no healthy (polyp-free) images being available to test whole-image classification. All code and results are shared publicly as a Colab notebook.

1. Introduction

Spotting a polyp during a colonoscopy is a good example of what researchers call anomaly detection: the model has to notice something that looks different from the healthy tissue around it, usually without having a huge labelled dataset of

that exact kind of anomaly to learn from [6]. A normal AI model can be trained to do this well if it is given thousands of images with the polyp area marked by hand. The problem is that this marking work is slow, needs a trained expert, and has to be redone for every new type of scan or hospital.

1.1. Why We Chose This Problem

CLIP [11] is a model trained on hundreds of millions of pictures paired with their captions from the internet. Because of this, it is already quite good at connecting an image to a piece of text, even for things it has never explicitly been trained on. That makes it tempting to just ask CLIP directly: does this patch of the image look more like “a polyp” or “healthy tissue”? No new labelled data needed.

The problem, pointed out clearly by [9], is that CLIP was never taught the difference between normal and abnormal. It learned to match pictures with captions describing what is in them, not whether something in them is wrong. So CLIP can recognise that it is looking at a colon, but it is much less reliable at telling a polyp apart from a normal fold of tissue. We checked this ourselves: plain, untrained CLIP scores 65–71 on pixel-level AUROC on our datasets, which sounds decent, but its Average Precision is only in the low 20s. In plain terms, it is not really finding the polyp; it is just guessing somewhere near it (see Section 5).

AA-CLIP [9] fixes this with a simple two-step idea. First, it teaches the text side of CLIP to treat the words “normal” and “anomalous” as two clearly separate ideas instead of two vague, similar ones. Second, it teaches the image side to match each small patch of the picture to whichever of those two ideas it looks more like. Both steps only train two small add-on modules; the original CLIP model is never changed. This keeps training fast and cheap, and is meant to protect everything CLIP already knows about images in general.

In this project we ran AA-CLIP end to end on a real, useful task: finding polyps in colonoscopy images. We tried to

test it properly rather than just get it running: we measured the untrained baseline ourselves instead of copying numbers from a paper, we used the exact same scoring script for the before and after results, and we tested the trained model on data it had never seen during training.

1.2. What We Set Out to Do

Our goals for this project, and what we actually delivered, were:

1. **Get the method working, start to finish.** We ran the full two-step AA-CLIP training process on colonoscopy images, keeping CLIP itself frozen and training only the two add-on modules, all inside a single Colab notebook that finishes in about an hour on a free GPU.
2. **Measure it honestly.** Instead of quoting a zero-shot score from someone else’s paper, we ran plain CLIP ourselves and scored it with the exact same code used to score the trained model afterward, so the improvement we report actually comes from the training and not from a mismatch in how the two were measured.
3. **Check that it actually generalises.** We trained the add-on modules using only the ClinicDB dataset, then tested on two other datasets the model had never seen. This was a real test that could have failed if the method did not work well.
4. **Be upfront about the limits.** We point out clearly where a result looks better than it really is (like the in-domain ClinicDB score) instead of only showing the good numbers.

The rest of this report is laid out as follows. Section 2 looks at earlier work on using CLIP for anomaly detection. Section 3 explains how AA-CLIP actually works. Section 4 covers the datasets, the metrics we used, and our exact settings. Section 5 shows and discusses our results. Section 6 looks at which parts of the method matter most. Section 7 lists what this study does not prove, and Section 8 wraps up with what we learned and what we would do next.

2. Related Work

The old way of doing anomaly detection. Older anomaly detection methods work by learning what “normal” looks like, then flagging anything that does not fit that pattern [6]. This works fine when there is plenty of normal data for the exact thing being checked, like one specific factory part. It works much worse when normal examples are scarce, which is often the case in medicine, since every new scan type or hospital starts with almost no labelled data.

CLIP. CLIP [11] was trained on huge numbers of images paired with their captions, using a method that teaches it to match the right image to the right caption out of many

options. Because of this training, CLIP can often recognise brand new categories it was never specifically taught, just by describing them in words. This has made it a popular starting point for many tasks, including trying to spot anomalies without labelled training data.

Using CLIP to find anomalies. WinCLIP [8] was one of the first attempts to use CLIP this way, comparing image features at several scales against handwritten prompts like “a photo of a damaged item.” Later work, such as VAND [5] and MVFA-AD [7], improved this by training small extra modules on the image side, but they still used CLIP’s original, untouched text descriptions as the target to match. Other methods, AnomalyCLIP [12] and AdaCLIP [4], instead trained new, learnable text prompts. This can work well, but it risks damaging the object knowledge CLIP already has, which can hurt performance on categories the model has never seen.

Why CLIP struggles with this in the first place. AA-CLIP [9] is, as far as we know, the first paper to clearly point out the actual root cause: CLIP’s text side never learned a clean boundary between “normal” and “abnormal” for a given object, so any method built on top of CLIP’s original text embeddings inherits that same confusion. AA-CLIP’s fix, training the text side and image side in two separate steps rather than touching CLIP’s core weights at all, is the method this report reproduces (Section 3). Because it never edits CLIP’s original text tokens or backbone, the idea is to gain the ability to spot anomalies without losing what CLIP already knew.

Medical datasets for this kind of task. There are established benchmarks for anomaly detection in medicine, covering brain MRI, liver CT, and eye scans through BMAD [1], and colonoscopy polyp datasets such as CVC-ClinicDB [3], CVC-ColonDB [2], and Kvasir-SEG [10], which we use in this report. These three polyp datasets come with a pixel-level mask marking exactly where the polyp is, but every image in them contains a polyp; there are no healthy images to compare against, which affects what we can and cannot measure (Section 4).

3. Method

We used the AA-CLIP method exactly as described by Ma *et al.* [9], without changing it, and pointed it at colonoscopy images. Before the technical details, here is the whole idea in one paragraph: CLIP already knows how to compare a picture to a sentence. We cut each colonoscopy image into small squares, and for every square we ask CLIP a simple question: does this look more like the word “normal” or the word “anomalous”? Whichever one

it matches better decides how much heat that square gets on the final map. The only thing we train is two small helper modules, one that sharpens what “normal” and “anomalous” mean to the text side, and one that teaches the image side to sort patches correctly. CLIP’s own brain never changes.

3.1. How the Matching Works

CLIP is really two models glued together: one that reads images, one that reads text. Neither of these two big models is touched in this project. An image is split into small square patches. Each patch is turned into a list of numbers (a vector), and so is each text prompt, such as “a photo of a colon” or “a photo of a broken colon.” Comparing two of these number-lists with a simple formula called cosine similarity tells you how close they are in meaning. A patch is called anomalous if it sits closer to the anomalous prompt’s numbers than to the normal prompt’s numbers. Doing this for every patch and stitching the results back into the shape of the original image gives the heatmap you see in Figure 2. There is no extra “detector” sitting on top; the matching itself is the whole detector.

3.2. The Two Small Helper Modules

Since CLIP’s own weights are frozen, AA-CLIP bolts on small trainable pieces, called adapters, onto the first few layers of both the text model and the image model. Think of an adapter as a small dial that nudges a feature slightly, rather than replacing it. For a layer’s output x^i , the adapter computes a nudged version of it,

$$x_{residual}^i = \text{Norm}(\text{Act}(W^i x^i)), \quad (1)$$

and then mixes that nudge back in with the original, untouched feature:

$$x_{enhanced}^i = \lambda x_{residual}^i + (1 - \lambda) x^i. \quad (2)$$

Here λ is just a small number, close to zero, that controls how much of the nudge gets kept. In short: the model mostly stays itself, with a small, controlled push toward noticing anomalies. This one detail turns out to matter a lot, we cover why in Section 6.

3.3. Step 1: Making “Normal” and “Anomalous” Mean Different Things

The first training step only touches the text side; the image side is left alone and used as a stable target to learn against. Prompts such as “a photo of [object]” and “a photo of broken [object]” get turned into two average number-lists, T_N for normal and T_A for anomalous. These two get adjusted so they line up well against real image features:

$$\begin{aligned} p_{cls} &= \text{CosSim}(V_{image}, [T_N, T_A]), \\ p_{seg}^o &= \text{CosSim}(V_{patch}, [T_N, T_A]). \end{aligned} \quad (3)$$

Training is guided by two kinds of feedback: was the whole image correctly called abnormal or not, and did the pixel-by-pixel prediction line up with where the polyp actually is:

$$\begin{aligned} \mathcal{L}_{cls} &= \text{BCE}(p_{cls}, y), \\ \mathcal{L}_{seg} &= \text{Dice}(p_{seg}, S) + \text{Focal}(p_{seg}, S), \\ \mathcal{L}_{align} &= \mathcal{L}_{cls} + \mathcal{L}_{seg}. \end{aligned} \quad (4)$$

On top of that, one more piece of feedback pushes T_N and T_A further apart on purpose, so they stop overlapping:

$$\mathcal{L}_{dis} = |\langle T_N, T_A \rangle|^2, \quad (5)$$

combined into one overall training signal, $\mathcal{L}_{total} = \mathcal{L}_{align} + \gamma \mathcal{L}_{dis}$. A simple way to picture this: put two labelled boxes, one for “normal” and one for “anomalous,” far apart on a table, before you try sorting anything into them. That is what this step does, entirely in the text side, before the image side is even involved.

3.4. Step 2: Teaching the Image Side Where to Look

Once the two text anchors from Step 1 are set, they are frozen in place, and now the image model gets its own adapters. The model pulls out features at four different zoom levels and adds them together:

$$\begin{aligned} V_{patch}^i &= \text{Proj}_i(F^i), \quad i \in \{1, 2, 3, 4\}, \\ V_{patch} &= \sum_{i=1}^4 V_{patch}^i. \end{aligned} \quad (6)$$

These combined features are compared against the two frozen text anchors the same way as in Equations (3)–(4), so a patch sitting on a polyp gets pulled toward “anomalous,” and a patch of healthy tissue gets pulled toward “normal.” At test time, this patch-by-patch score, stretched back out to the size of the original photo, is the final heatmap. Nothing more is added on top of it.

3.5. Why Two Separate Steps Instead of One

It would be simpler to train both sides together at the same time. AA-CLIP avoids this on purpose. Training both sides at once tends to make the model overreact to the word “anomalous” and forget the general object-knowledge CLIP started with, which hurts it on categories it has never seen before. Finishing the text side first, then locking it in place while the image side trains, keeps that general knowledge safe on one side while the other side is being changed. This order, text first and then image, rather than training both together, is the one design choice we think matters the most, and it is the one we kept exactly as the original authors described it.

Dataset	Images	Used for	Seen while training?
CVC-ClinicDB	612	Train + test	Yes
CVC-ColonDB	380	Test only	No
Kvasir-SEG	1,000	Test only	No

Table 1. The three datasets used in this study. We trained the add-on modules using CVC-ClinicDB only. CVC-ColonDB and Kvasir-SEG were never shown to the model during training, so they test whether the method actually generalises.

4. Experimental Setup

4.1. Datasets

We used three public colonoscopy datasets, each one with a mask that marks exactly where the polyp is in every image (Table 1): CVC-ClinicDB (612 images) [3], CVC-ColonDB (380 images) [2], and Kvasir-SEG (1,000 images) [10].

We trained the model on CVC-ClinicDB only and tested it on all three. Doing it this way keeps two things separate: how well the model does on data it has already seen, versus how well it does on data it has never seen before. CVC-ColonDB and Kvasir-SEG are never touched during training, so any improvement measured on them cannot just be the model memorising the training set. We treat the CVC-ClinicDB score as a sanity check rather than a real result, because in the code we built on, training and testing on ClinicDB both use the same list of images, which makes its “after” score look better than it should (more on this in Section 7).

4.2. How We Measured Success

We used two scores, and both were calculated by the exact same piece of code for the before (untrained) and after (trained) model, so the comparison is fair:

- **Pixel AUROC.** Line up every pixel in the image from “most likely a polyp” to “least likely.” Do the pixels that are actually part of the polyp tend to land near the top of that list? That is what this score checks. A score of 100 would mean a perfect ranking; 50 is no better than a coin flip.
- **Pixel AP.** Out of all the pixels the model actually flags as a polyp, how many of them are really part of the polyp? This is a tougher, more honest score than AUROC, because a model can rank things in roughly the right order while still being wrong about most of the individual pixels, especially when the polyp is a small part of a much bigger image, which is exactly our case.

We did not measure whether a whole image contains a polyp at all, because we could not: every single image in all three datasets already has a polyp in it. There is no “healthy” image in this data to compare against, so that

Setting	Value	Note
Base model	CLIP ViT-L/14-336px	Frozen
Training images	All 612 (ClinicDB)	Full set
Text adapter	5 rounds	Step 1
Image adapter	10 rounds	Step 2
Batch size	2	Limited by GPU memory
Image size	336 px	Matches base model
Memory setting	<code>expandable_segments</code>	Avoids running out of memory

Table 2. The settings used for our main results. AA-CLIP keeps two full copies of the ViT-L/14 model in memory at once, which, together with optimizer memory, uses up almost all 15 GB of a free Tesla T4 GPU before batch size is even a factor; a batch size of 8 or 16 simply runs out of memory.

question simply cannot be asked here. The code prints a zero for this, and that zero just means “not applicable,” not that the model failed. One more useful fact about both scores we did use: they only care about the order the model ranks pixels in, not the raw numbers it outputs. That is convenient for us, since the untrained model’s raw scores and the trained model’s raw scores are not even on the same scale, so a fair side-by-side comparison would otherwise be tricky.

4.3. How We Set It Up

We used OpenCLIP’s ViT-L/14-336px model as the frozen base model, which matches the image size it was originally trained at. Every original CLIP weight stayed frozen the whole time; only the small adapters and a few extra projector layers were trained. Table 2 lists our exact settings.

Training and testing run as two separate scripts (`train.py` and `test.py`), both called from a single Colab notebook, so the exact same evaluation code scores the untrained baseline and the trained model. The whole process, from setting up the environment to downloading the base model to training both adapters to testing on all three datasets, takes about an hour on a free Google Colab GPU. If Colab disconnects partway through, training picks back up from the last finished round instead of starting over, since a checkpoint is saved after each round.

We also built a faster, lighter version of this setup for quick testing (32 training examples per round, 3 rounds for the text adapter, 5 rounds for the image adapter, 224 px images instead of 336 px). We discuss this separately in Section 6 rather than treating it as a main result, since it trades off some accuracy for speed.

5. Results and Discussion

5.1. The Numbers

Table 3 shows pixel-level AUROC and AP for plain CLIP and for the AA-CLIP-trained model, on all three datasets,

Dataset	Pixel AUC		Pixel AP	
	Before	After	Before	After
ClinicDB [†]	65.1	99.4	23.4	95.7
ColonDB	71.0	92.8	22.5	71.6
Kvasir	67.6	97.2	34.5	90.7

Table 3. Pixel-level AUROC and AP (%) before and after training the add-on modules. Training only used CVC-ClinicDB; ColonDB and Kvasir were never seen during training. [†]ClinicDB is trained and tested on the same images, so treat it as a sanity check, not proof that the method generalises (Section 7).

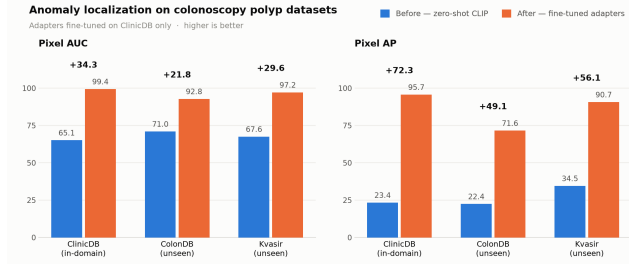


Figure 1. Pixel AUC (left) and Pixel AP (right), before and after training, across all three datasets. The add-on modules were trained on ClinicDB only, so ColonDB and Kvasir show real, unseen-data performance.

using the same scoring script both times. Figure 1 shows the same numbers as a chart.

On the two datasets the model had never seen during training, pixel AUROC went from 67.6–71.0 up to 92.8–97.2, a jump of about 22 to 30 points, and pixel AP went from 22.5–34.5 up to 71.6–90.7, a jump of about 49 to 56 points, which is more than triple where it started. The AP jump matters more than the AUROC jump. A score in the low 20s is close to what you would get from a blurry, spread-out guess, while a score above 70 means the model is putting most of its confidence right on the actual polyp. As expected, ClinicDB shows the biggest jump of all (+34.3 AUC, +72.3 AP), but we do not treat that number as our main result, for the reasons explained in Section 7.

5.2. What the Model Is Actually Looking At

Figure 2 shows heatmaps for four ClinicDB images, comparing what the untrained and trained models pay attention to, next to the original image and the real polyp location. The pattern is the same across all four examples: untrained CLIP’s heat spreads out over bright spots and edges that have nothing to do with the polyp, like shiny reflections on wet tissue, the dark opening further down the colon, and the edges of the picture itself. This lines up with the low AP scores in Table 3: the model is reacting to contrast, not to disease.

After training, the model’s attention consistently pulls

Anomaly localization — before vs. after fine-tuning

Red = higher predicted anomaly score · samples from ClinicDB

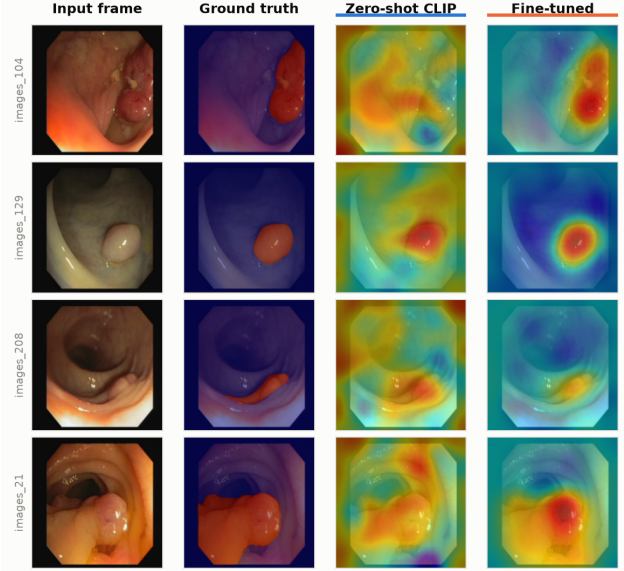


Figure 2. Heatmaps before and after training, on four ClinicDB images. Red means the model thinks a polyp is more likely there. Untrained CLIP’s attention is scattered across bright spots and edges; the trained model’s attention pulls in tight around the actual polyp.

in around the polyp itself, and the rest of the image cools down. The clearest example is the second row (images_129), where a vague warm smear covering most of the frame turns into one sharp hotspot sitting right on the lesion. This kind of change, attention tightening up rather than just shifting around, is exactly what we would expect from the disentangle loss in Equation (5): once “normal” and “anomalous” are pulled apart in the text space, a patch only needs to move a little to clearly land on one side or the other, which sharpens the resulting map.

5.3. What We Take Away From This

Two things stand out. First, ClinicDB, ColonDB, and Kvasir come from different scopes, different lighting, and different patients, so the improvement on ColonDB and Kvasir cannot just be the model memorising specific images from training; it points to the model actually learning a more general idea of what a polyp looks like. Second, Kvasir improved slightly more than ColonDB (+56.1 vs. +49.1 AP), which fits with Kvasir’s polyps generally being larger and higher-contrast, closer in appearance to ClinicDB’s, while ColonDB’s polyps tend to be smaller and flatter. This suggests the smaller gain on ColonDB is more about how different that dataset looks, not a weakness in the training method itself.

What was changed	Adding it	Removing it
Plain linear adapter	Makes it worse	–
Residual adapter (image)	Fixes & improves	Drops back down
Residual adapter (text)	Improves further	Text stays confused
Disentangle loss	Best overall	Anchors stay too close

Table 4. A plain-language summary of the ablation reported in the AA-CLIP paper [9]. Adding a basic linear layer directly into the model, without the residual blending step, actually made zero-shot performance worse. That is the key reason the blending formula in Equation (2) exists: it is what lets the model gain anomaly-awareness without losing what it already knew.

6. Ablation Study

Here we look at two things: which parts of AA-CLIP actually matter, based on tests the original authors ran, since repeating their full ablation ourselves was outside what we could do in this project; and what changes when we use a faster, cheaper training setup ourselves.

6.1. Which Parts of the Method Matter (From the Original Paper)

Ma *et al.* [9] started from a simpler baseline (a basic linear layer added to the image side, with CLIP’s original, untouched text descriptions, similar to VAND [5]) and then added each of AA-CLIP’s pieces one at a time to see what each one actually buys you. Table 4 sums up the pattern of what they found, and it is the reason we kept every one of these pieces in our own reproduction.

The most useful thing to take from this table is the first row: just bolting on a plain linear layer, without the residual blend from Equation (2), actually made the model worse than doing nothing at all. That told us the blending step is not a small detail, it is the reason the whole approach works, which is why we did not try simplifying it in our own reproduction.

6.2. One Training Step vs. Two

The original authors also tried training the text and image sides together in one combined step, instead of one after the other (this is also how AdaCLIP [4] works). They found this made the model worse on both metrics, and their visualisations showed the text side losing track of object categories much faster when trained this way. This matches our own choice, explained in Section 3.5, to keep the text anchors frozen while training the image side, rather than updating both parts of the model at the same time.

6.3. Our Own Test: Full Training vs. a Faster Version

Separately from the tests above, we built a second, quicker training setup for fast iteration while developing our note-

Setting	Full run (main)	Fast run
Training images	All 612	32 per round
Text adapter	5 rounds	3 rounds
Image adapter	10 rounds	5 rounds
Image size	336 px	224 px
Time taken	About 1 hour	Faster (not timed exactly)

Table 5. The full setup we used for our reported results, next to a quicker setup built purely for faster development iterations.

book, alongside the full version used for our main results (Table 2). Table 5 compares the two.

We report the full run as our main result because that is the run for which we have complete, saved before-and-after numbers on all three datasets (Table 3). The fast run trades away most of the training images, a shorter training schedule, and a smaller image size (224 px instead of the base model’s native 336 px) in exchange for speed. Based on the resolution drop alone, we would expect it to lose some sharpness on small, flat polyps, which lines up with what Ma *et al.* note about the trade-off between image size and fine detail. We did not finish a full scoring run of this fast setup in time for this report, so we are not including made-up numbers for it here; instead we list it as a limitation (Section 7) and as something to finish as future work (Section 8).

7. Limitations

We want to be upfront about what this study does and does not prove.

- **The ClinicDB score looks better than it should.** The code we built on trains and tests on the same list of ClinicDB images, so that score is measured on data the model has already seen and should not be treated as proof the method generalises. CVC-ColonDB and Kvasir-SEG, which the model never saw during training, are the numbers that actually show real-world performance.
- **Only one type of scan.** Everything in this report is about colonoscopy polyps. It says nothing about whether this same training approach would work on other kinds of medical images, such as brain MRI or liver CT scans, which the original AA-CLIP paper does test but which we did not attempt ourselves.
- **No whole-image classification.** Every image in our three datasets already contains a polyp, so we could only test *where* a polyp is, never *whether* an image has one at all. A real clinical tool would also need healthy images to learn and test that second question.
- **Just one run.** Every number in Table 3 comes from a single training run. We did not repeat it multiple times, so we cannot report how much the results might vary from run to run.

- **Smaller image size.** We trained and tested at 336 pixels for our main results, and at 224 pixels for the faster setup discussed in Section 6.3. Both are smaller than what a clinical-grade system would likely use, and small, flat polyps in particular could lose some detail because of this.
- **Not a medical device.** This is a research reproduction meant to test a published method fairly. It has not been checked or approved for actual diagnostic use, and should not be treated as one.

8. Conclusion and Future Work

We reproduced AA-CLIP [9], a method that trains two small add-on modules to make CLIP better at spotting anomalies, and tested it on finding polyps in colonoscopy images using a fair before-and-after setup: the same frozen base model, the same three datasets, and the same scoring code, run once before any training and once after. Training these two small modules, while leaving CLIP itself completely untouched, took pixel-level AP on datasets the model had never seen from the low 20s up to 71–91, more than tripling it, and the heatmaps show the model’s attention moving away from bright, distracting spots and settling right onto the polyp. The whole process takes about an hour on a free GPU, so anyone can run it again or build on it.

We see this as a genuinely useful middle ground, not a replacement for a fully supervised model trained on thousands of hand-labelled images for one specific dataset. It does not match what that kind of model could achieve, but it gets a large part of the way there without needing any new pixel-level labels for a new dataset, which is exactly the kind of cost that makes labelled medical data so hard to come by in the first place.

Looking at the limitations in Section 7, we see five clear next steps: run the whole pipeline several times so we can report a range instead of one number; keep a separate test split of ClinicDB aside so its score becomes a real result instead of a sanity check; try the same approach on other kinds of scans, like brain MRI or liver CT, to see if the same generalisation holds outside colonoscopy; train at full image size on a bigger GPU and measure how much detail the smaller size actually costs us; and add healthy, polyp-free images so the model can also learn to answer whether an image is anomalous at all, which would also let us properly finish scoring the faster training setup from Section 6.3 instead of only describing it.

9. Data Availability

All three datasets used in this study are public: CVC-ClinicDB and CVC-ColonDB are shared by their original authors [2, 3], and Kvasir-SEG is shared by Simula Research Laboratory [10]. Each dataset has its own license,

held by its original authors, which should be checked before any reuse. The AA-CLIP method we reproduced is described in the original paper [9], and the authors have released their own code publicly.

Our own reproduction, including the training and testing notebook, the settings used for CVC-ClinicDB, CVC-ColonDB, and Kvasir-SEG, the small code fixes needed to run the original code on Colab, and the result figures shown in this report, is publicly available here:

https://github.com/mohit6603/Medical_Image_Anomaly_Detection

The repository includes a one-click Google Colab notebook that runs the whole pipeline, from setup and downloading the base model through training both adapters to scoring the before and after results on all three datasets, in about an hour on a free GPU.

References

- [1] Jinan Bao, Hanshi Sun, Hanqiu Deng, Yinsheng He, Zhaoxiang Zhang, and Xingyu Li. Bmad: Benchmarks for medical anomaly detection, 2024.
- [2] Jorge Bernal, Javier Sánchez, and Fernando Vilarino. Towards automatic polyp detection with a polyp appearance model. *Pattern Recognition*, 45(9):3166–3182, 2012.
- [3] Jorge Bernal, F Javier Sánchez, Gloria Fernández-Esparrach, Debora Gil, Cristina Rodríguez, and Fernando Vilarino. WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation vs. saliency maps from physicians. *Computerized Medical Imaging and Graphics*, 43:99–111, 2015.
- [4] Yunkang Cao, Jiangning Zhang, Luca Frittoli, Yuqi Cheng, Weiming Shen, and Giacomo Boracchi. Adapclip: Adapting clip with hybrid learnable prompts for zero-shot anomaly detection. In *European Conference on Computer Vision*, pages 55–72. Springer, 2025.
- [5] Xuhai Chen, Yue Han, and Jiangning Zhang. April-gan: A zero-/few-shot anomaly classification and segmentation method for cvpr 2023 vand workshop challenge tracks 1&2: 1st place on zero-shot ad and 4th place on few-shot ad. *arXiv preprint arXiv:2305.17382*, 2023.
- [6] Tharindu Fernando, Harshala Gammulle, Simon Denman, Sridha Sridharan, and Clinton Fookes. Deep learning for medical anomaly detection—a survey. *ACM Computing Surveys (CSUR)*, 54(7):1–37, 2021.
- [7] Chaoqin Huang, Aofan Jiang, Jinghao Feng, Ya Zhang, Xinchao Wang, and Yanfeng Wang. Adapting visual-language models for generalizable anomaly detection in medical images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11375–11385, 2024.
- [8] Jongheon Jeong, Yang Zou, Taewan Kim, Dongqing Zhang, Avinash Ravichandran, and Onkar Dabeer. Winclip: Zero-/few-shot anomaly classification and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19606–19616, 2023.

- [9] Wenxin Ma, Xu Zhang, Qingsong Yao, Fenghe Tang, Chenxu Wu, Yingtai Li, Rui Yan, Zihang Jiang, and S. Kevin Zhou. AA-CLIP: Enhancing zero-shot anomaly detection via anomaly-aware CLIP. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4744–4754, 2025.
- [10] Konstantin Pogorelov, Kristin Ranheim Randel, Carsten Griwodz, Sigrun Losada Eskeland, Thomas de Lange, Dag Johansen, Concetto Spampinato, Duc-Tien Dang-Nguyen, Mathias Lux, Peter Thelin Schmidt, et al. Kvasir: A multi-class image dataset for computer aided gastrointestinal disease detection. In *Proceedings of the 8th ACM on Multimedia Systems Conference*, pages 164–169, 2017.
- [11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021.
- [12] Qihang Zhou, Guansong Pang, Yu Tian, Shibo He, and Jiming Chen. Anomalyclip: Object-agnostic prompt learning for zero-shot anomaly detection. *arXiv preprint arXiv:2310.18961*, 2023.